AI Dev Assistant

Continuous Edge-Case & Business-Logic Checker for VS Code

Project Expo — MVP Technical Reference

Prepared by Tamil · Solo Project

1. Project Overview

A VS Code extension that continuously watches code as it's written and flags edge cases and business-logic issues — combining fast local static analysis with LLM-assisted reasoning for the kind of bugs pattern-matching alone can't catch. Suggest-only: no auto-fix, no auto-apply.

1.1 Problem

Standard linters (ESLint, TS compiler) catch syntax and type errors, not domain-logic mistakes — e.g. a discount function that allows negative prices, or a state transition that skips a required step. Those bugs pass every existing static check and only surface at runtime or in production.

1.2 Scope boundary (explicit, not a cop-out)

"Detect all edge cases in all business logic" is not achievable — correctness requires knowing intent, and intent lives in a spec or a person's head, not in code. This MVP scopes to: JS/TS and Java, function-level analysis, two detection tiers (deterministic + LLM-reasoning), suggest-only output. This boundary is a design decision, not a limitation to hide.

2. Architecture

2.1 High-Level Architecture

Five components: the editor surface, the extension host (runs Tier-1 locally), an in-process orchestrator (context building + caching), a swappable LLM provider interface, and the two output surfaces — Diagnostics and Chat. No separate backend process; everything but the external LLM call runs inside the extension host.

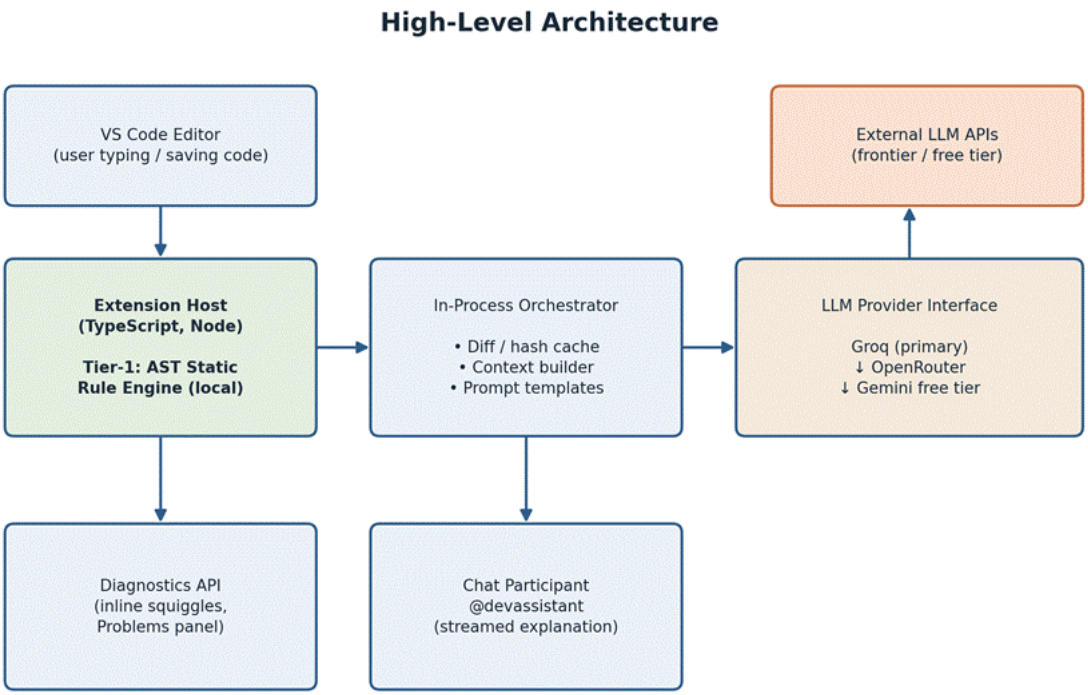

Figure 1 — High-Level Architecture

2.2 Low-Level Architecture

Module-level breakdown, updated to add Java support. A new LanguageAdapter dispatches by file type ahead of Tier-1; the existing TypeScriptAdapter + JS/TS RuleEngine path is untouched, and a parallel JavaAdapter + Java RuleEngine path is added alongside it. Both feed the same DiagnosticMapper output shape, so Tier-2 (orchestrator, cache, provider chain) and the entire output layer require zero changes — they already operate on normalized text, not on language-specific ASTs.

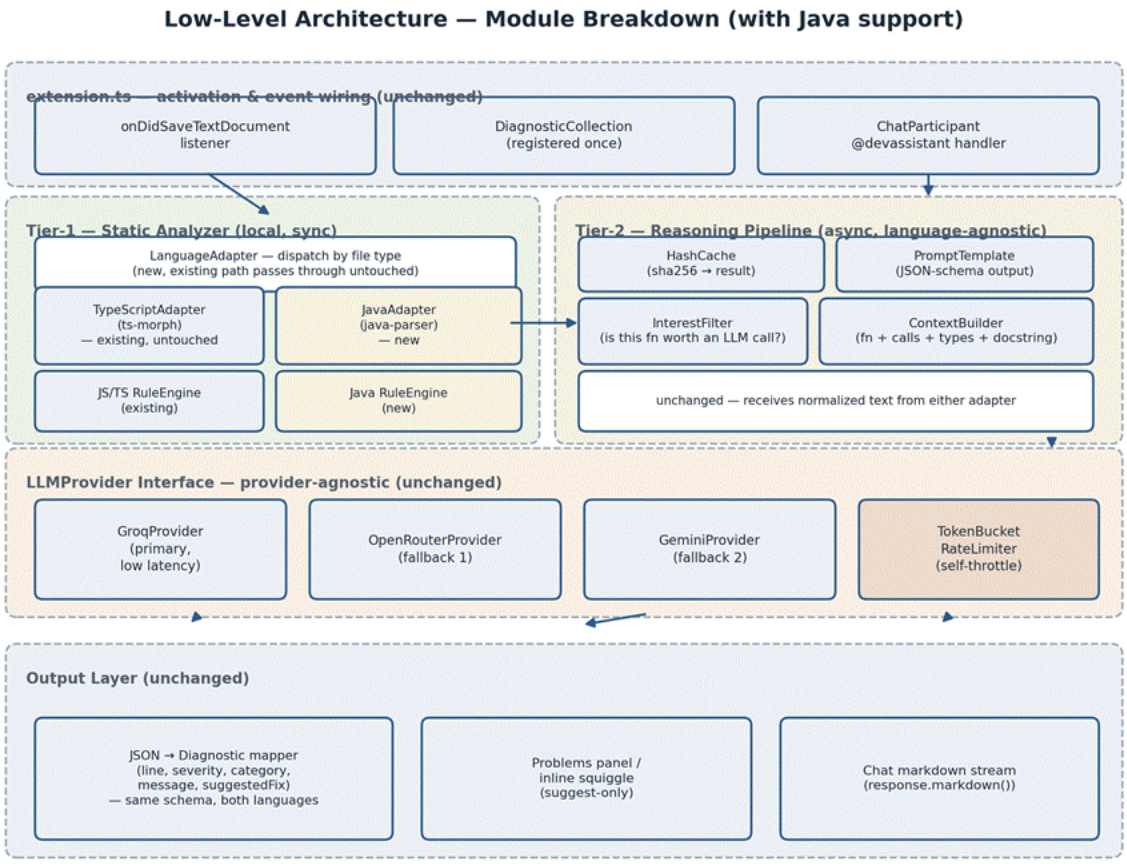

Figure 2 — Low-Level Architecture (module breakdown, with Java support)

2.3 End-to-End Flow

Sequence from a single save event through to feedback in the editor. Tier-1 results (step 3) return before Tier-2 is even invoked — perceived responsiveness never depends on network/LLM latency. Steps 5–8 (cache check → provider call → store) are the cost- and latency-control layer.

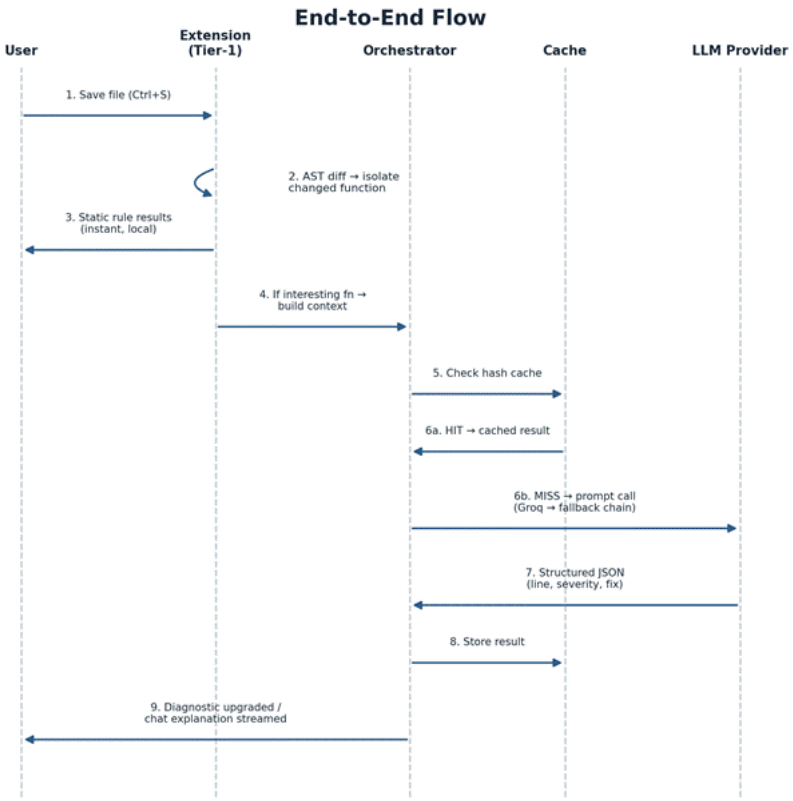

Figure 3 — End-to-End Sequence Flow

3. Detection Tiers

3.1 Tier-1 — Deterministic Static Analysis

AST pattern checks, no LLM, no network, instant. Zero hallucination risk — this is the reliability layer. Two rule sets now run behind the LanguageAdapter, selected by file type; neither touches the other.

JS/TS RuleEngine (existing, unchanged):

- Null/undefined access without a guard
- Array/string bounds — index used without a length check
- Division / modulo by zero
- Unhandled promise rejection, missing try/catch around await
- == vs === and implicit type coercion
- Off-by-one loop bounds
- Unvalidated function parameters used without a range/type check

Java RuleEngine (new):

- Null dereference without a guard — NullPointerException-prone access
- Array index used without a bounds check — ArrayIndexOutOfBoundsException risk
- Division / modulo by zero
- Checked exception paths with no try/catch and no throws declaration
- == used for object/String comparison instead of .equals()

- Off-by-one loop bounds
- Unvalidated method parameters used without a range/null check

3.2 Tier-2 — LLM-Assisted Reasoning

Fires only for functions the heuristic flags as "interesting" (business-logic keywords: calculate, validate, apply, transition, process, etc.), and only after a Tier-1 pass and cache miss. Needs an intent signal — a docstring/comment — or it will produce plausible-but-wrong flags. Runs identically for JS/TS and Java — ContextBuilder output is plain text regardless of source language, so this tier needed no changes to support Java.

- Value-invariant bugs — e.g. a discount that can push price negative
- State-invariant bugs — e.g. an illegal status transition (refunded → shipped)
- Any logic violation that requires understanding stated intent, not just syntax

4. Tech Stack & Tools

Layer	Choice	Notes
Extension runtime	TypeScript, VS Code Extension API	Unchanged — mandatory, extension host is Node/Electron only
JS/TS AST parsing	ts-morph	Unchanged — wraps TS Compiler API
Java AST parsing	java-parser (npm, ANTLR-based)	New — pure JS/TS, no JVM/JDK dependency, keeps in-process architecture intact
Tier-1 rules	Hand-written, per language, over each adapter's AST	JS/TS set unchanged; Java set new, same output shape
HTTP client	Native fetch (Node 18+)	Unchanged
LLM providers	Groq (primary) → OpenRouter → Gemini free tier	Unchanged — language-agnostic, Groq chosen primary for latency
Hashing / cache key	Node crypto (sha256)	Unchanged — content-hash keyed in-memory cache
Cache store	In-memory Map	Unchanged — no Redis/persistence needed for MVP
Diagnostics UI	vscode.languages.createDiagnosticCollection	Unchanged — native squiggles + Problems panel
Chat UI	vscode.chat.createChatParticipant + response.markdown()	Unchanged
Packaging	vsce → .vsix	Unchanged — sideload for demo
Extension tests	@vscode/test-electron	Unchanged, optional
Prompt dev/testing	Standalone Node script (outside VS Code)	Unchanged — extend fixtures with a Java scenario

5. Rate-Limit Mitigation (Groq)

Layered, and none of it costs perceived performance — the design goal is fewer real calls plus a UX that never waits on the network in the first place.

- Content-hash cache — unchanged functions never trigger a re-call, largest single reduction in call volume
- InterestFilter heuristic — skip trivial getters/setters, only logic-bearing functions reach Tier-2 at all
- Client-side token-bucket — self-throttle against Groq's known rate limit before hitting a 429, proactive not reactive
- Provider fallback on limit — switch to OpenRouter/Gemini immediately rather than backoff-retry the same provider
- Tier-1 always renders first — primary feedback loop never depends on Tier-2 network latency at all
- Demo safety net — pre-run demo scenarios once before presenting so live runs are cache hits, zero live API dependency

6. Demo Scenarios

Three scenarios, each proving a different capability — pure static, value-invariant reasoning, state-invariant reasoning. ~90 seconds of live typing each.

6.1 Scenario A — Tier-1, no AI

getUserEmail(user) returns user.profile.email without a guard — profile can be undefined. Squiggle appears before the line is finished typing. Establishes credibility before AI enters the demo.

6.2 Scenario B — Tier-2, value-invariant

applyDiscount(price, discountPercent) has no bound check — discountPercent > 100 produces a negative price. Type-checks fine, no syntax error; only catchable by reasoning about the comment + domain intent.

6.3 Scenario C — Tier-2, state-invariant

updateOrderStatus allows any OrderStatus → any OrderStatus, including refunded → shipped. Both values are individually valid — it's the transition that's illegal. Different bug shape from B, shows the tool isn't a one-trick pattern matcher.

6.4 Scenario D — Java, optional

A Java method comparing two String objects with == instead of .equals() — catchable by the new Java RuleEngine (Tier-1, instant). Optional addition to show the Java path live; not required to make the core demo arc work.

7. Development Flow

Recommended build order — pipeline logic before extension wiring, so prompt iteration doesn't require reopening the Extension Development Host each time.

- Phase 1 — Standalone Node script: context builder → prompt template → provider call → JSON parse, tested against the 3 demo scenarios
- Phase 2 — Tier-1 rule engine over ts-morph, tested independently of any LLM call
- Phase 3 — Extension scaffold: package.json contribution points, onDidSaveTextDocument wiring, DiagnosticCollection
- Phase 4 — Wire Tier-1 into the extension host, verify squiggles render on save
- Phase 5 — Wire Tier-2 orchestrator (cache, InterestFilter, provider fallback chain) into the extension
- Phase 6 — Chat participant for “explain more”, streamed markdown
- Phase 7 — Rehearse the 3 demo scenarios end-to-end, pre-warm cache, package as .vsix

- Phase 8 — Java support: add LanguageAdapter dispatch, JavaAdapter (java-parser) + Java RuleEngine behind it; existing JS/TS path untouched throughout

8. Developer Reference

8.1 Suggested repo structure

- src/extension.ts — activation, event wiring only (unchanged)
- src/tier1/languageAdapter.ts — new, dispatches by document languageId
- src/tier1/astParser.ts, ruleEngine.ts, diagnosticMapper.ts — existing JS/TS path, untouched
- src/tier1/javaAdapter.ts, javaRuleEngine.ts — new, mirror the JS/TS path's shape
- src/tier2/ — interestFilter.ts, contextBuilder.ts, promptTemplate.ts, hashCache.ts (unchanged, language-agnostic)
- src/providers/ — llmProvider.ts (interface), groq.ts, openRouter.ts, gemini.ts, rateLimiter.ts (unchanged)
- src/chat/ — chatParticipant.ts (unchanged)
- scripts/pipeline-dev.ts — standalone Node harness for prompt iteration; extend fixtures with a Java sample
- test/ — demo scenario fixtures, now including one Java file alongside the JS/TS ones

8.2 Config / environment

- API keys read from environment variables, never hardcoded — GROQ_API_KEY, OPENROUTER_API_KEY, GEMINI_API_KEY
- Rate-limit thresholds (requests/min per provider) kept as named constants, not magic numbers, so they're easy to adjust after checking each provider's current published limits

8.3 Things to verify before demo day

- Run all 3 (or 4, with Java) scenarios once beforehand to warm the cache
- Confirm fallback chain actually triggers — temporarily invalidate the Groq key and verify OpenRouter picks up
- Confirm the 5s timeout path shows the “Tier-1 only” fallback message instead of hanging
- Test on a machine without internet — Tier-1 should still work fully offline, for both languages
- Confirm saving a .java file routes through JavaAdapter and a .ts/.js save still behaves exactly as before — the two paths must not interfere
- Confirm java-parser needs no JVM/JDK installed on the demo machine — it's a pure JS parser, this should already be true, verify once

9. Next Steps

- Build Phase 1 (standalone pipeline script) and validate against the 3 demo scenarios
- Scaffold the extension (Phase 3) once the pipeline output quality is confirmed
- Revisit this document as decisions evolve — it's the single source of truth for the build